OBJECT-ORIENTED PROGRAMMING (OOPs)

1. Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects and classes**. It is used to design software by representing real-world entities as objects containing data and functions.

In traditional procedural programming, a program is mainly organized around functions and procedures. In OOP, the primary focus is on objects that combine **data and the operations performed on that data**.

For example, in a student management system, a student can be represented as an object containing information such as:

- Name
- Roll number
- Age
- Marks

The object can also have functions such as:

- Display details
- Calculate percentage
- Check result

OOP makes programs easier to organize, maintain, reuse, and modify.

Popular programming languages that support OOP include:

- C++
- Java
- Python
- C#
- JavaScript

---

2. Basic Concepts of OOP

The main concepts of Object-Oriented Programming are:

1. Class

2. Object
3. Encapsulation
4. Abstraction
5. Inheritance
6. Polymorphism

These concepts are also commonly referred to as the fundamental principles or pillars of OOP.

---

3. Class

A **class** is a blueprint or template used to create objects.

It defines the properties and behaviors that objects of that class will have.

For example, consider a class named Student.

It may contain:

Data members:

- Name
- Roll number
- Marks

Member functions:

- Display()
- CalculateMarks()

Example in C++

```
class Student
{
  public:
    string name;
    int rollNo;

    void display()
```

```
    {
        cout << name << " " << rollNo;
    }
};
```

Here, Student is a class containing data members and a member function.

A class itself does not represent a specific student. It provides a structure from which individual student objects can be created.

---

4. Object

An **object** is an instance of a class.

If a class is considered a blueprint, an object is the actual entity created using that blueprint.

For example:

Student s1;

Student s2;

Here, s1 and s2 are objects of the Student class.

Each object can have its own values.

For example:

s1.name = "Rahul";

s1.rollNo = 101;

s2.name = "Aman";

s2.rollNo = 102;

Both objects belong to the same class but contain different data.

Objects allow programs to represent real-world entities in a structured manner.

---

5. Encapsulation

Encapsulation is the process of combining data and the functions that operate on that data into a single unit.

In OOP, a class is an example of encapsulation because it combines data members and member functions.

Encapsulation also helps in **data hiding**, where sensitive data can be protected from direct access.

For example:

```
class Account
{
  private:
    double balance;

  public:
    void setBalance(double b)
    {
      balance = b;
    }

    double getBalance()
    {
      return balance;
    }
};
```

Here, balance is private and cannot be directly accessed from outside the class.

It can only be accessed through public functions such as setBalance() and getBalance().

Advantages of Encapsulation

- Provides data security
- Prevents unauthorized access
- Makes code easier to maintain
- Reduces complexity

- Provides controlled access to data

---

6. Abstraction

Abstraction means hiding unnecessary implementation details and showing only the essential information to the user.

A common real-world example is an ATM.

When using an ATM, the user performs operations such as:

- Entering PIN
- Checking balance
- Withdrawing money

The user does not need to know the internal process through which the bank verifies the transaction.

Similarly, in programming, abstraction hides the internal implementation and exposes only the required functionality.

In C++, abstraction can be achieved using:

- Classes
- Access specifiers
- Abstract classes
- Pure virtual functions

Example

```
class Car
{
  public:
    void start()
    {
      cout << "Car started";
    }
};
```

The user only needs to call start(). The internal working of the engine is not required to use the function.

---

7. Inheritance

Inheritance is a mechanism through which one class can acquire the properties and functions of another class.

The existing class is called the **base class** or **parent class**, while the new class is called the **derived class** or **child class**.

For example, suppose we have a class Animal.

```
class Animal
{
  public:
    void eat()
    {
      cout << "Eating";
    }
};
```

A Dog class can inherit from Animal.

```
class Dog : public Animal
{
  public:
    void bark()
    {
      cout << "Barking";
    }
};
```

Now a Dog object can use both functions:

```
Dog d;
```

d.eat();

d.bark();

Thus, inheritance promotes **code reusability**.

Types of Inheritance

The major types of inheritance are:

1. Single inheritance
2. Multiple inheritance
3. Multilevel inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

Single Inheritance

One derived class inherits from one base class.

A → B

Multilevel Inheritance

A class is derived from another derived class.

A → B → C

Multiple Inheritance

One derived class inherits from multiple base classes.

A + B → C

Hierarchical Inheritance

Multiple derived classes inherit from the same base class.

A → B
A → C

---

8. Polymorphism

The word **polymorphism** means "many forms."

In OOP, polymorphism allows the same function, operator, or interface to behave differently in different situations.

There are two major types of polymorphism:

1. Compile-time polymorphism
2. Run-time polymorphism

8.1 Compile-Time Polymorphism

Compile-time polymorphism is determined during compilation.

It can be achieved using:

- Function overloading
- Operator overloading

Function Overloading

Function overloading means having multiple functions with the same name but different parameters.

Example:

```
class Calculator
{
  public:

    int add(int a, int b)
    {
      return a + b;
    }

    int add(int a, int b, int c)
    {
      return a + b + c;
    }
};
```

Here, both functions are named add() but have different numbers of parameters.

The compiler determines which function should be called based on the arguments.

8.2 Run-Time Polymorphism

Run-time polymorphism occurs when the method to be executed is determined during program execution.

It is commonly achieved using **function overriding and virtual functions**.

For example:

```
class Animal
{
  public:
    virtual void sound()
    {
      cout << "Animal sound";
    }
};

class Dog : public Animal
{
  public:
    void sound()
    {
      cout << "Bark";
    }
};
```

The derived class provides its own implementation of the sound() function.

This allows the same function call to produce different behavior depending on the object.

---

9. Constructors

A **constructor** is a special member function that is automatically called when an object is created.

The constructor has the same name as the class and does not have a return type.

Example:

```
class Student
{
  public:

    Student()
    {
      cout << "Object created";
    }
};
```

When an object is created:

```
Student s;
```

the constructor is automatically executed.

Types of Constructors

Common types include:

1. Default constructor
2. Parameterized constructor
3. Copy constructor

Default Constructor

A constructor that does not take any parameters is called a default constructor.

Parameterized Constructor

A constructor that accepts parameters is called a parameterized constructor.

Example:

```
Student(int r)
{
```

```
    rollNo = r;
}
```

Copy Constructor

A copy constructor creates a new object by copying the values of another object.

---

10. Destructor

A **destructor** is a special member function that is automatically called when an object is destroyed.

In C++, a destructor is represented using the ~ symbol.

Example:

```
class Student
{
    public:

      ~Student()
      {
         cout << "Object destroyed";
      }
};
```

Destructors are useful for releasing resources used by an object.

For example, they can be used to release dynamically allocated memory or close resources.

---

11. Access Specifiers

Access specifiers define how members of a class can be accessed.

The three main access specifiers in C++ are:

1. Public
2. Private
3. Protected

Public

Members declared as public can be accessed from outside the class.

Private

Private members can be accessed only within the class.

Protected

Protected members can be accessed within the class and by derived classes.

A simple comparison is:

Access Specifier	Same Class	Derived Class	Outside Class
Public	Yes	Yes	Yes
Protected	Yes	Yes	No
Private	Yes	No	No

---

12. Advantages of OOP

Object-Oriented Programming provides several advantages.

1. Code Reusability

Inheritance allows existing code to be reused instead of writing it again.

2. Data Security

Encapsulation and access specifiers help protect data from unauthorized access.

3. Modularity

Large programs can be divided into smaller classes and objects.

4. Easy Maintenance

Changes can be made to individual classes without modifying the entire program.

5. Flexibility

Polymorphism allows the same interface to be used for different implementations.

6. Real-World Representation

Objects can represent real-world entities such as students, employees, cars, accounts, and products.

---

13. Applications of OOP

OOP is widely used in software development.

Some important applications include:

- Software development
- Game development
- Web applications
- Mobile applications
- Banking systems
- Enterprise applications
- Simulation systems
- Artificial intelligence and machine learning software
- GUI applications

For example, in a banking application, objects can represent customers, accounts, transactions, and employees.

Similarly, in a game, objects can represent players, enemies, weapons, vehicles, and game environments.

---

14. Difference Between Procedural Programming and OOP

Procedural Programming	Object-Oriented Programming
Focuses mainly on functions	Focuses on objects
Data may be exposed to functions	Data can be protected using encapsulation
Less suitable for very large systems	Suitable for large and complex systems
Code reuse is comparatively limited	Supports inheritance and reuse
Follows a procedure-oriented approach	Follows an object-oriented approach
Example: C	Examples: C++, Java, Python

---

15. Conclusion

Object-Oriented Programming is an important programming paradigm that organizes software around classes and objects. It provides important concepts such as **encapsulation, abstraction, inheritance, and polymorphism**.

OOP improves code organization, reusability, security, flexibility, and maintainability. By representing real-world entities as objects, complex software systems can be designed in a structured and understandable way.

Languages such as C++, Java, Python, and C# widely use object-oriented concepts. Due to its advantages, OOP has become one of the most widely used approaches for developing modern software applications.